Lab Exercise 20

App Deployment Using ECS And ECR

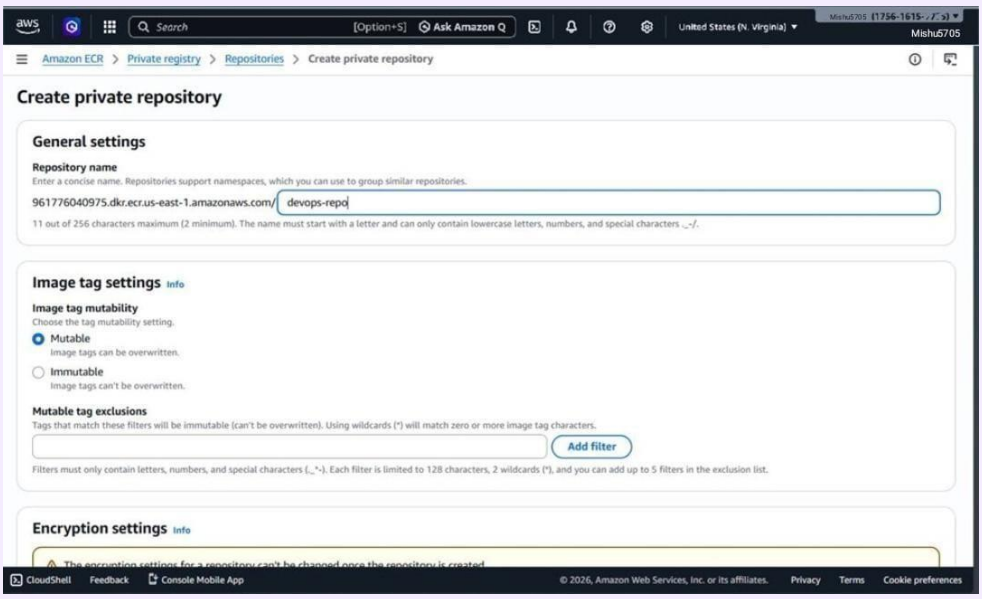
Name- Manish kumar

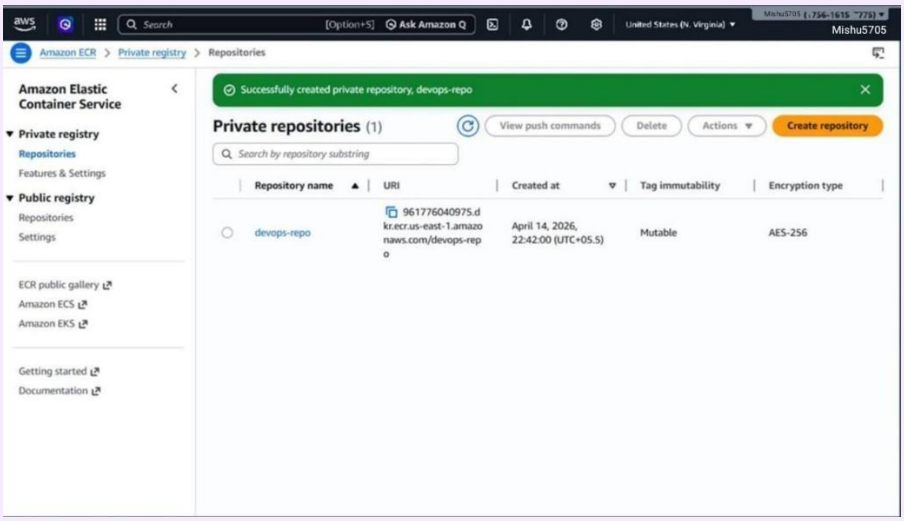
Batch-2

SAP ID- 500119723

Step 1: Create ECR Repository

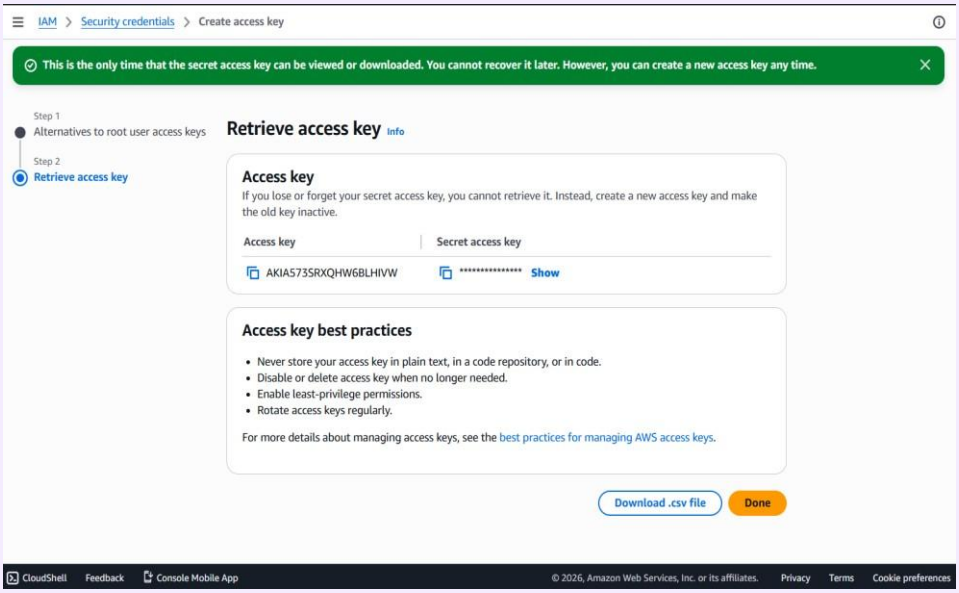
- Go to AWS Console
- Search **Elastic Container Registry (ECR)**
- Click **Create Repository**
- Give name (example: my-web-app)
- Click **Create**





Step 2: Configure aws

Create access key




```
C:\Users\hp>aws configure
AWS Access Key ID [****************SR2E]: AKIARONSQQITOGVOEY7D
AWS Secret Access Key [****************I49u]: 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
Default region name [us-east-1]:
Default output format [json]:

C:\Users\hp>
```

Step 3: Authenticate Docker to ECR

Run this :

```
aws ecr get-login-password --region ap-south-1 | docker login --username AWS --passwordstdin

099695493670.dkr.ecr.ap-south-1.amazonaws.com
```

```
C:\Users\hp>aws ecr get-login-password --region ap-south-1 | docker login --username AWS --password-stdin 099695493670.dkr.ecr.ap-south-1.amazonaws.com
Login Succeeded

C:\Users\hp>
```

Step 4: Build Docker Image Run

this :

```
docker build -t devops-ecr .
```

```
 => => transferring dockerfile: 93B                                                                                    0.0s
 => [internal] load metadata for docker.io/library/httpd:latest                                                       10.9s
 => [auth] library/httpd:pull token for registry-1.docker.io                                                           0.0s
 => [internal] load .dockerignore                                                                                      0.4s
 => => transferring context: 2B                                                                                        0.0s
 => [1/3] FROM docker.io/library/httpd:latest@sha256:bdba5c86022f2d6ad0783168b07fe80541fe45e97b434b13ba8c959a5050fb6b  26.1s
 => => resolve docker.io/library/httpd:latest@sha256:bdba5c86022f2d6ad0783168b07fe80541fe45e97b434b13ba8c959a5050fb6b   0.4s
 => => sha256:bdba5c86022f2d6ad0783168b07fe80541fe45e97b434b13ba8c959a5050fb6b 10.14kB / 10.14kB                       0.0s
 => => sha256:6b1468bed5457ca32b91740328647bfbc00dfe4a3d6f8dc81ee70a387435d8bb 2.09kB / 2.09kB                         0.0s
 => => sha256:e48c16e072a6f60e551a262fec6b96d54ba13974acc2112d3c683d65be2deafc 7.99kB / 7.99kB                         0.0s
 => => sha256:5435b2dcdf5cb7faa0d5b1d4d54be2c72a776fab9a605336f5067d6e9ecb5976 29.70MB / 29.70MB                      22.7s
 => => sha256:8261ede62f8c1c85c3e4b2ae3a9ae5a586b04fcc61b60c110ee65c9c523b49c7 146B / 146B                             8.0s
 => => sha256:4f4fb700ef54461cfa02571ae0db9a0dc1e0cdb5577484a6d75e68dc38e8acc1 32B / 32B                               7.6s
 => => sha256:a838a4be55ced5f2d7208c56a3a13ecbd15110574ee338337c50737aef163fdf 2.00MB / 2.00MB                        11.6s
 => => sha256:61e1798e23ba12f14505e225391338db290ee6cecfb6321c6c6b92ac8b546dcb 13.45MB / 13.45MB                      24.9s
 => => sha256:4e51448030490b8604d7662b05848e89e3563fe3b955159f7ec54d4d48a4893f 289B / 289B                            12.7s
 => => extracting sha256:5435b2dcdf5cb7faa0d5b1d4d54be2c72a776fab9a605336f5067d6e9ecb5976                              1.6s
 => => extracting sha256:8261ede62f8c1c85c3e4b2ae3a9ae5a586b04fcc61b60c110ee65c9c523b49c7                              0.0s
 => => extracting sha256:4f4fb700ef54461cfa02571ae0db9a0dc1e0cdb5577484a6d75e68dc38e8acc1                              0.0s
 => => extracting sha256:a838a4be55ced5f2d7208c56a3a13ecbd15110574ee338337c50737aef163fdf                              0.2s
 => => extracting sha256:61e1798e23ba12f14505e225391338db290ee6cecfb6321c6c6b92ac8b546dcb                              0.5s
 => => extracting sha256:4e51448030490b8604d7662b05848e89e3563fe3b955159f7ec54d4d48a4893f                              0.0s
 => [internal] load build context                                                                                      0.5s
 => => transferring context: 554B                                                                                      0.0s
 => [2/3] WORKDIR /usr/local/apache2/htdocs/                                                                           0.2s
 => [3/3] COPY                                                                                                         0.1s
 => exporting to image                                                                                                 0.1s
 => => exporting layers                                                                                                0.1s
 => => writing image sha256:af22f887c7af5788a8ae5e6448980692db46e836eba1fbea20d4ccc28b819687                           0.0s
 => => naming to docker.io/library/devops-ecr                                                                          0.0s
```

Step 5: Tag Docker Image Run
this :

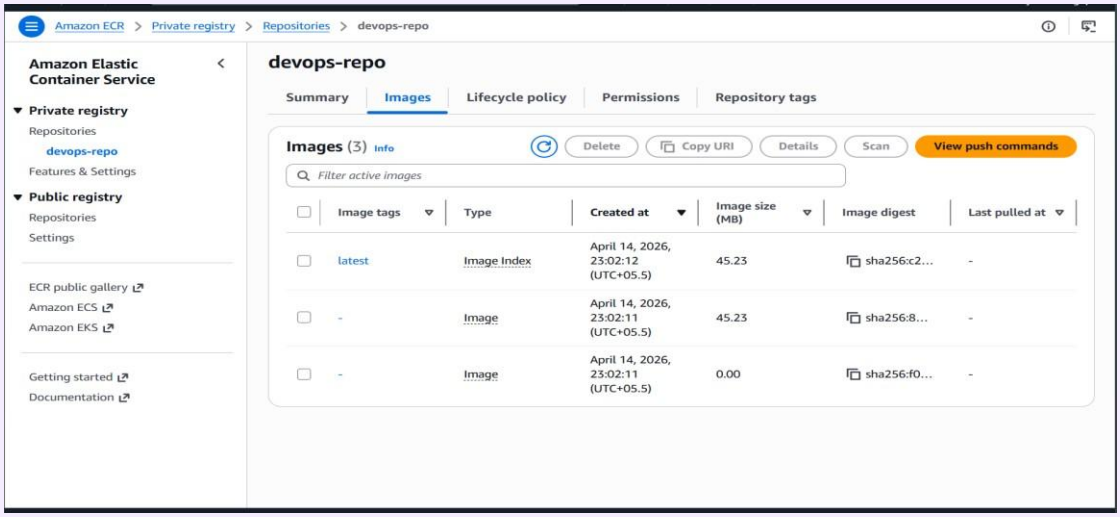
```
docker tag devops-ecr:latest 099695493670.dkr.ecr.ap-south-

1.amazonaws.com/devopsecr:demo
```

Step 6 : Push Docker Image

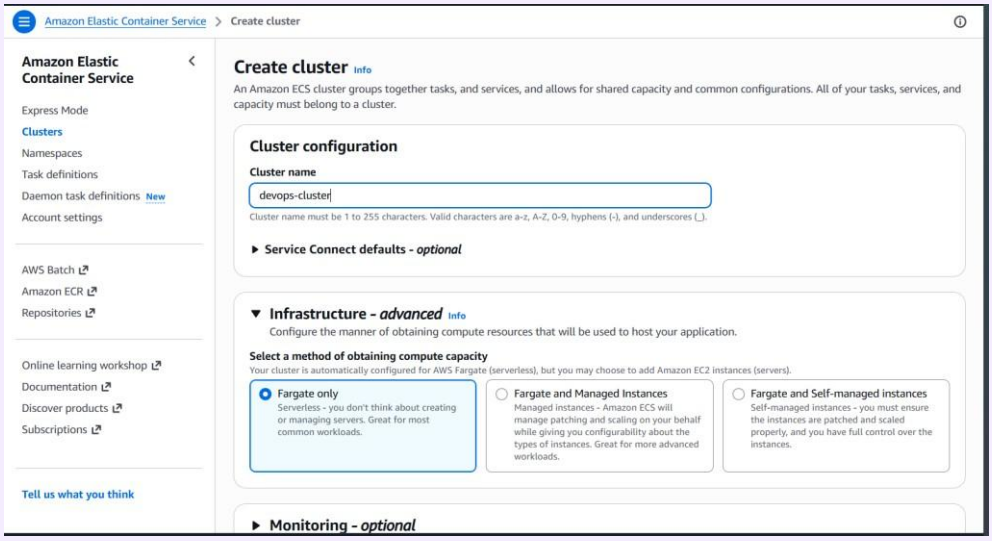
Run this :

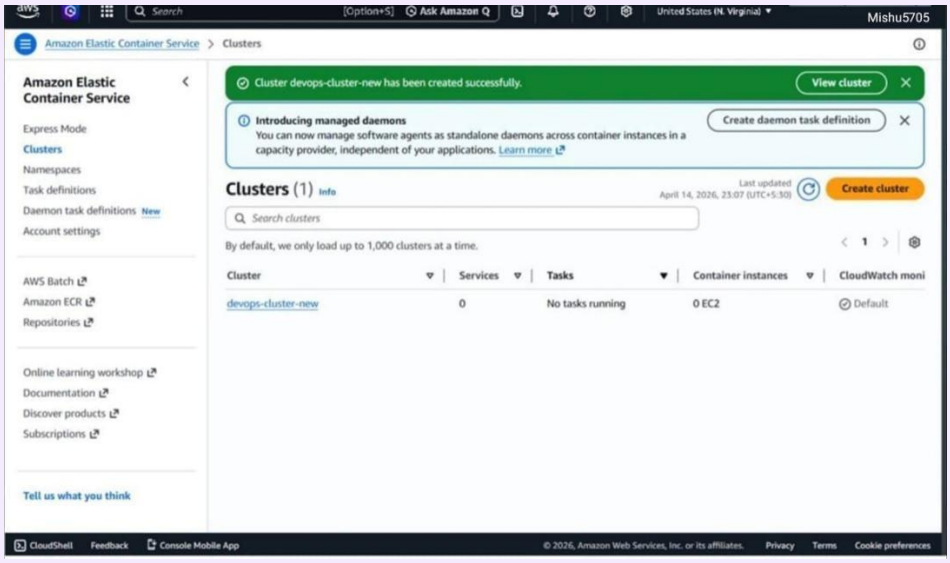
```
docker push 099695493670.dkr.ecr.ap-south-1.amazonaws.com/devops-repo:latest
```



Step 7: Create Cluster

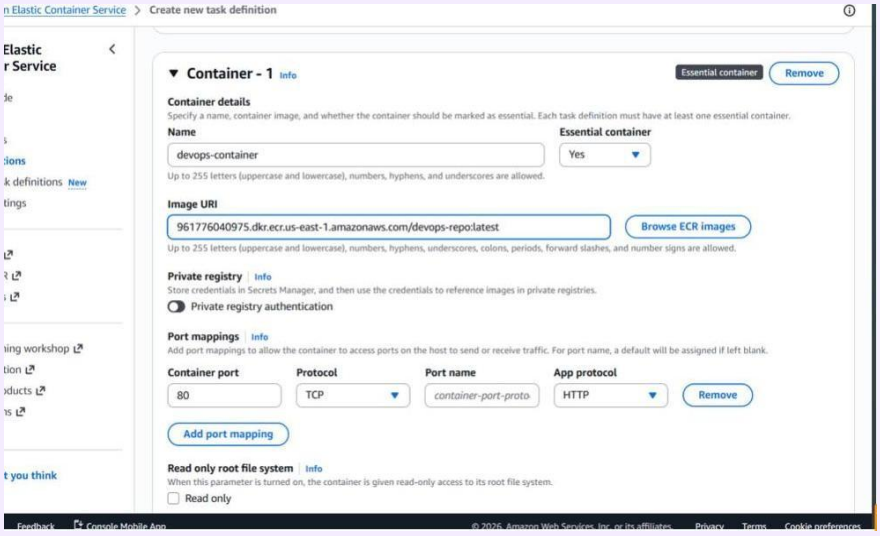
1. Go to ECS
2. Click **Create Cluster**
3. Select **Fargate (Serverless)**
4. Cluster Name: devops-cluster-new
5. Click Create





Step 8: Create Task Definition

1. Launch Type → Fargate
2. Container Image → Paste ECR URI



Step 9 Create Service

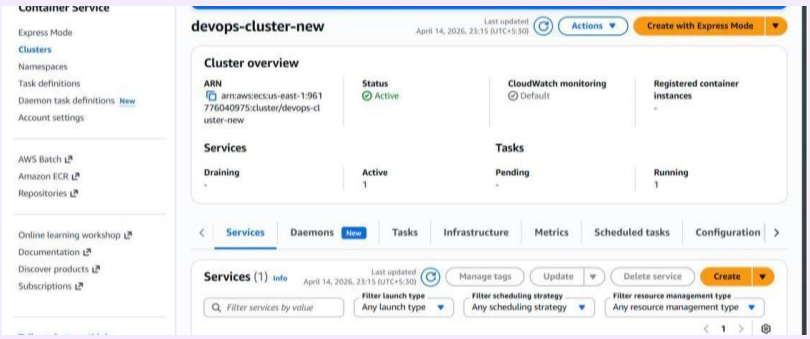
1. Go to Cluster
2. Click **Create Service**

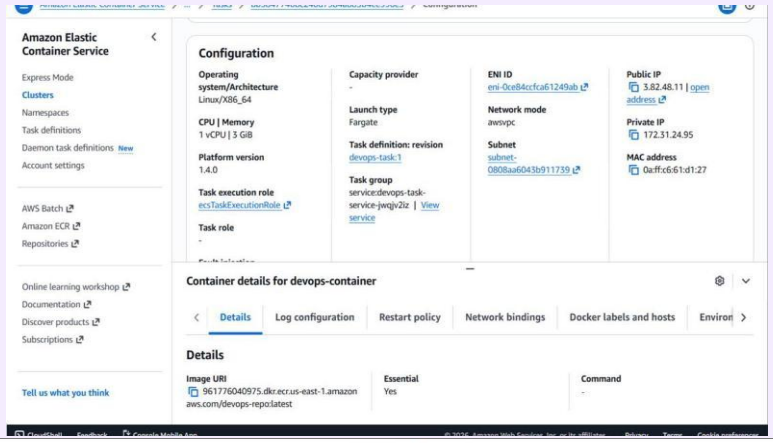
Step 10: Turn on all traffic in security broke

Step 11 : Verify Deployment

1. Go to Load Balancer
2. Copy DNS Name
3. Paste in Browser

http://3.82.48.11/





This is DevOps Engineering Course

click here for home page